

Task for today ASG

1. Create one VPC in N. Virginia region.

Task 1 — Create VPC in N. Virginia

First check the top-right AWS region is:

United States (N. Virginia) — us-east-1

Then go to:

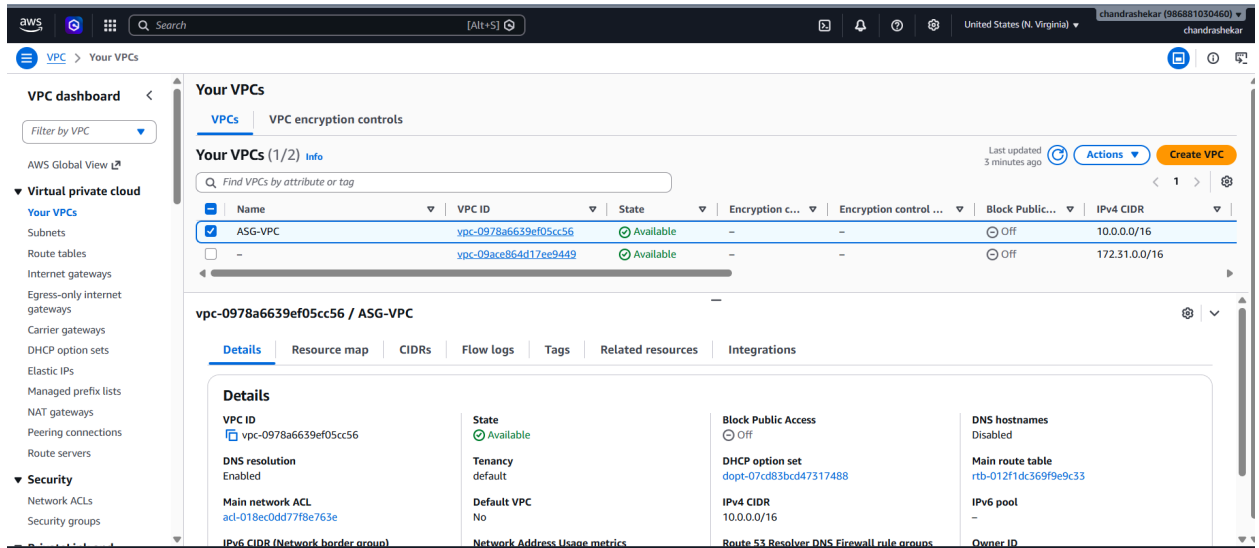
AWS Console → Search VPC → VPC → Your VPCs → Create VPC

Choose **VPC only**.

Enter:

Setting	Value
Name tag	ASG-VPC
IPv4 CIDR manual input	10.0.0.0/16
IPv6 CIDR block	No IPv6 CIDR block
Tenancy	Default

Leave the remaining options at their defaults.



2. Create two subnets: one public subnet and one private subnet.

Go to **VPC** → **Subnets** → **Create subnet** and enter:

Setting	Value
VPC	ASG-VPC
Subnet name	ASG-Public-Subnet
Availability Zone	us-east-1a
IPv4 subnet CIDR	10.0.1.0/24

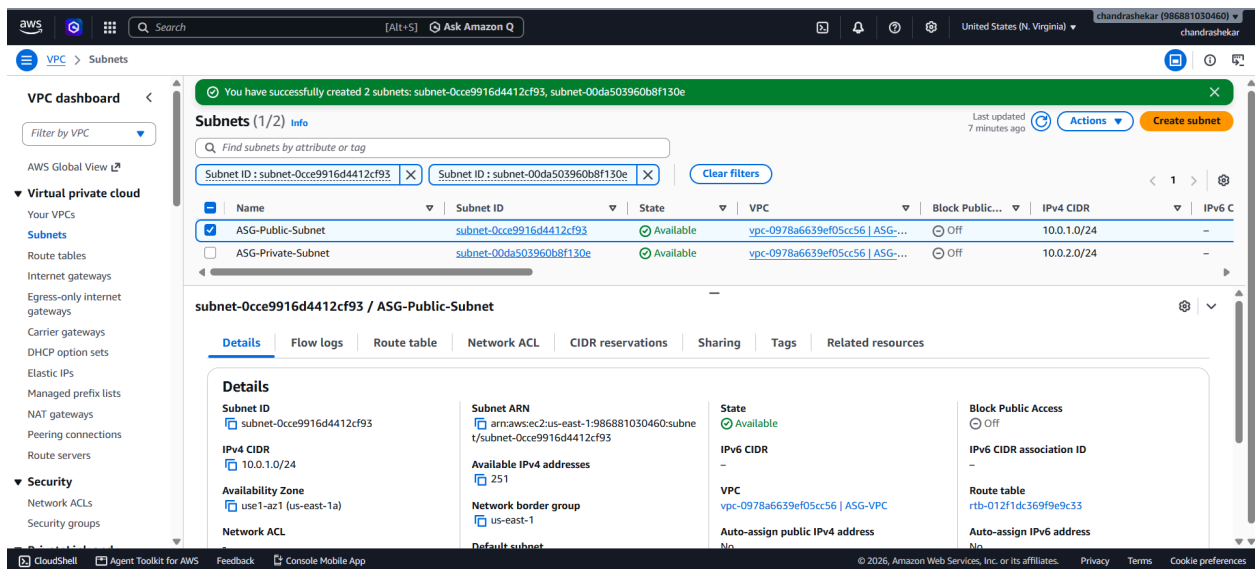
Click **Create subnet**.

For your **ASG-VPC**, enter:

Subnet	Availability Zone	IPv4 CIDR
ASG-Public-Subnet	us-east-1a	10.0.1.0/24
ASG-Private-Subnet	us-east-1b	10.0.2.0/24

So to create the first subnet, click **Add new subnet**, enter the private subnet details, then click **Create subnet** once.

Using different AZs (1a and 1b) is also useful because later when we work with the **Application Load Balancer and Auto Scaling**, we'll already have subnets across different Availability Zones.

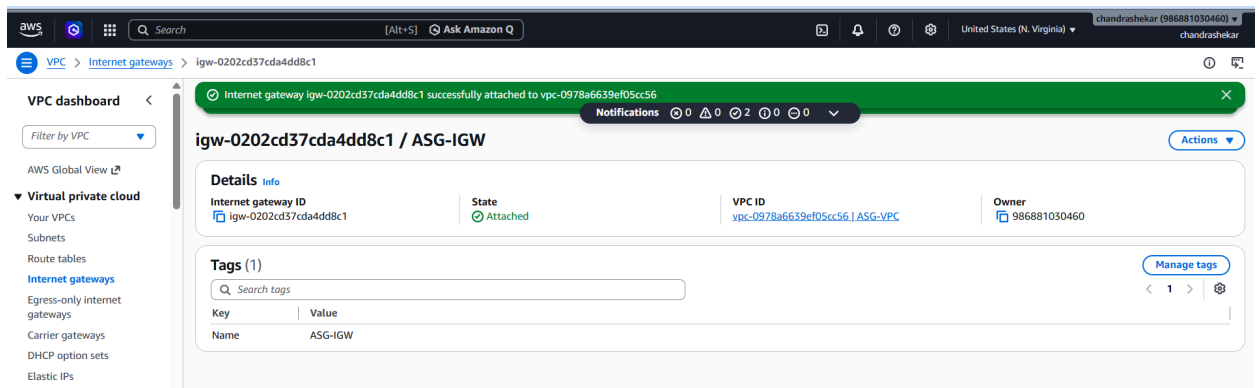


3. Attach an IGW to the VPC.

On the left side, click **Internet gateways**.

Then:

1. Click **Create internet gateway**.
2. Name: **ASG-IGW**
3. Click **Create internet gateway**.
4. After creation, click **Actions** → **Attach to a VPC**.
5. Select your **ASG-VPC**.
6. Click **Attach internet gateway**.



4. Create one public route table (RT) and one private route table.

the **Public Route Table** first.

1. Left menu → **Route tables**
2. Click **Create route table**
3. Name: **ASG-Public-RT**
4. VPC: select **ASG-VPC**
5. Click **Create route table**

ou are already on the **Routes** tab.

1. Click **Edit routes**.
2. Click **Add route**.
3. Destination:
0.0.0.0/0
4. Target: select **Internet Gateway**.
5. Select **ASG-IGW**.
6. Click **Save changes**.

You should then have two routes:

VPC CIDR → local

0.0.0.0/0 → ASG-IGW

Updated routes for rtb-09ef072657e7050b7 / ASG-Public-RT successfully

rtb-09ef072657e7050b7 / ASG-Public-RT

Details

Route table ID: rtb-09ef072657e7050b7

VPC: vpc-0978a6639ef05cc56 | ASG-VPC

Main: No

Owner ID: 986881030460

Explicit subnet associations: -

Edge associations: -

Routes (2)

Destination	Target	Status	Propag...	Route Origin
0.0.0.0/0	igw-0202c...	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

Click **Subnet associations** (next to Routes).

Click **Edit subnet associations**.

Tick **ASG-Public-Subnet** only.

Do not tick **ASG-Private-Subnet**.

You have successfully updated subnet associations for rtb-09ef072657e7050b7 / ASG-Public-RT.

rtb-09ef072657e7050b7 / ASG-Public-RT

Details

Route table ID: rtb-09ef072657e7050b7

VPC: vpc-0978a6639ef05cc56 | ASG-VPC

Main: No

Owner ID: 986881030460

Explicit subnet associations: subnet-0cce9916d4412cf93 / ASG-Public-Subnet

Edge associations: -

Subnet associations

Subnet	Association
subnet-0cce9916d4412cf93 / ASG-Public-Subnet	☒
subnet-0cce9916d4412cf93 / ASG-Private-Subnet	☐

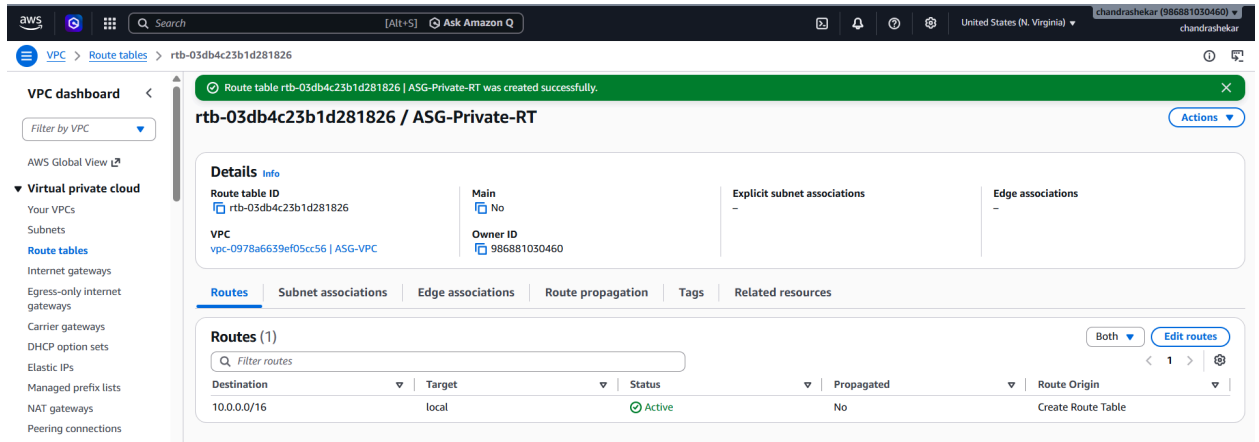
Your setup now is:

ASG-Public-Subnet → ASG-Public-RT → ASG-IGW → Internet ✓

Next: Create the Private Route Table

Go to **Route tables** → **Create route table** and enter:

- **Name:** ASG-Private-RT
- **VPC:** ASG-VPC
- Click **Create route table**



Route table rtb-03db4c23b1d281826 / ASG-Private-RT

Details

Route table ID rtb-03db4c23b1d281826	Main No	Explicit subnet associations -	Edge associations -
VPC vpc-0978a6639ef05cc56 ASG-VPC	Owner ID 986881030460		

Routes (1)

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

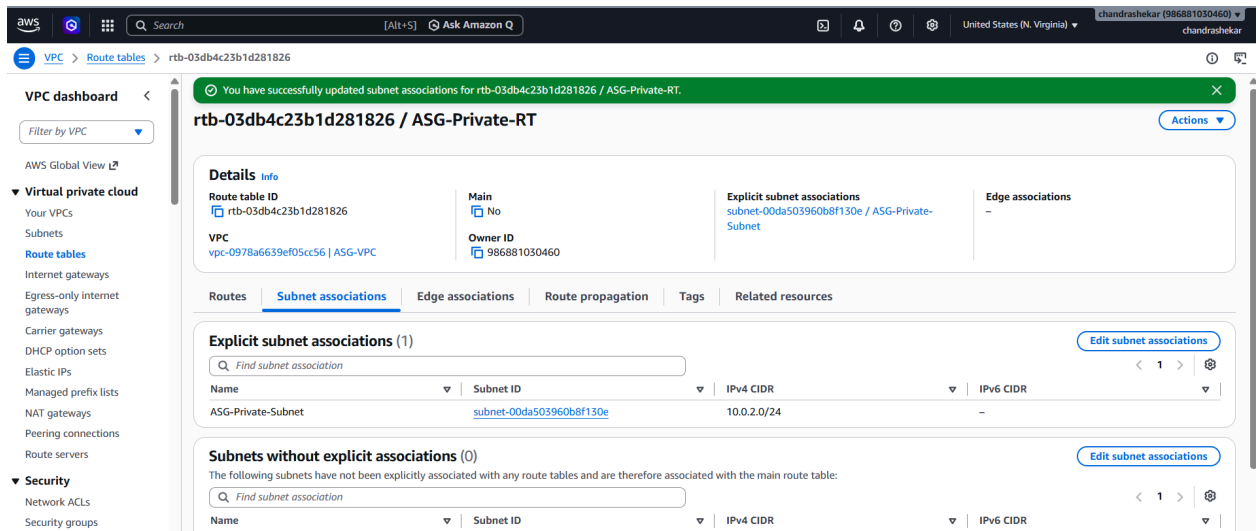
Click **Subnet associations**.

Click **Edit subnet associations**.

Select only **ASG-Private-Subnet**.

Do **not** select **ASG-Public-Subnet**.

Click **Save associations**.



Route table rtb-03db4c23b1d281826 / ASG-Private-RT

Details

Route table ID rtb-03db4c23b1d281826	Main No	Explicit subnet associations subnet-00da503960b8f130e / ASG-Private-Subnet	Edge associations -
VPC vpc-0978a6639ef05cc56 ASG-VPC	Owner ID 986881030460		

Subnet associations

Explicit subnet associations (1)

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
ASG-Private-Subnet	subnet-00da503960b8f130e	10.0.2.0/24	-

Subnets without explicit associations (0)

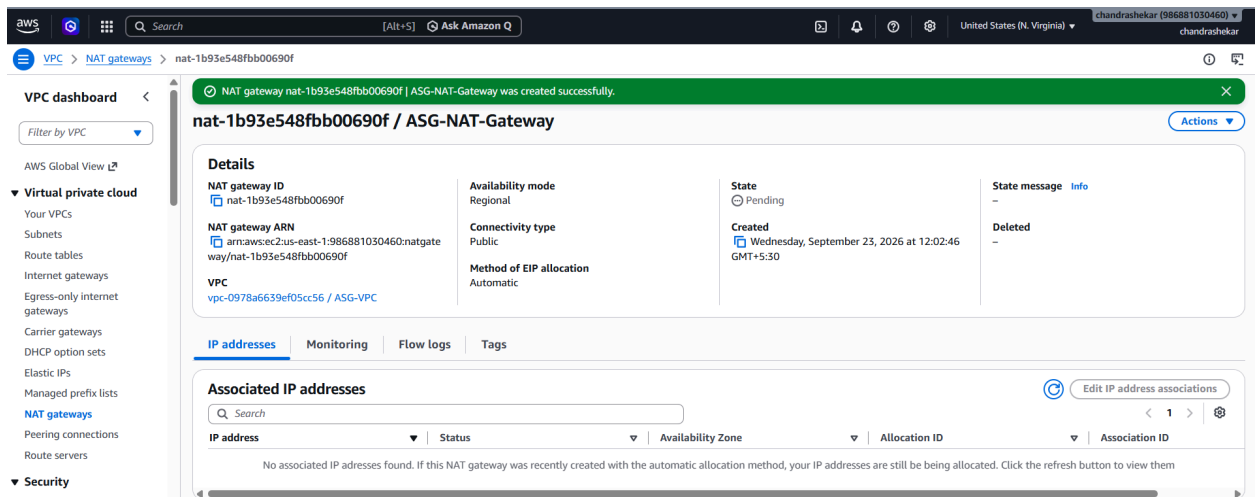
The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
------	-----------	-----------	-----------

5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

From the left menu:

1. Click NAT gateways.
2. Click Create NAT gateway.
3. Name: **ASG-NAT-Gateway**
4. Subnet: select **ASG-Public-Subnet** ⚠️ — not the private subnet.
5. Connectivity type: **Public**
6. Under Elastic IP allocation, click Allocate Elastic IP.
7. Click Create NAT gateway.



The architecture will become:

Internet

|

IGW

|

Public Subnet

|

NAT Gateway

|

Private Route Table

|

Private Subnet

next step: configure the Private Route Table.

Go to **VPC** → **Route tables** → select **ASG-Private-RT** → **Routes** → **Edit routes** → **Add route**.

Set:

- **Destination:** **0.0.0.0/0**
- **Target:** **NAT Gateway**
- Select your **ASG-NAT-Gateway**
- Click **Save changes**

After saving, your private route table should contain:

Dest inati on	Target
10.0.0.0/16	local
0.0.0.0/0	ASG-NAT-Gateway

This means:

Private EC2 → **Private Route Table** → **NAT Gateway** → **Internet Gateway** → **Internet**

The screenshot displays the AWS Management Console interface for a specific route table. At the top, a green banner indicates that routes for the route table were updated successfully. Below this, the route table's name 'rtb-03db4c23b1d281826 / ASG-Private-RT' is shown. The 'Details' section provides information about the route table ID, VPC, and owner ID. The 'Routes' tab is active, showing a list of two routes. The first route is for the destination '0.0.0.0/0' with the target 'nat-1b93e...' and is marked as 'Active'. The second route is for the destination '10.0.0.0/16' with the target 'local' and is also marked as 'Active'. The console interface includes a sidebar with navigation options like 'VPC dashboard', 'Virtual private cloud', and 'Security'.

Destination	Target	Status	Propag...	Route Origin
0.0.0.0/0	nat-1b93e...	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

6. Create two instances, one in the public subnet and one in the private subnet.

Go to **EC2** → **Instances** → **Launch instances**.

Use:

- Name: **ASG-Public-EC2**
- AMI: **Ubuntu Server**
- Instance type: **t2.micro** / free-tier eligible option shown in your account
- Key pair: use your existing key pair or create one
- VPC: **ASG-VPC**
- Subnet: **ASG-Public-Subnet**
- Auto-assign public IP: **Enable**

For Security Group allow:

- SSH — TCP 22 — **My IP** (preferred)
- HTTP — TCP 80 — **0.0.0.0/0**

Next: Create Private EC2

Click **Launch instances** again and use almost the same settings, but change these:

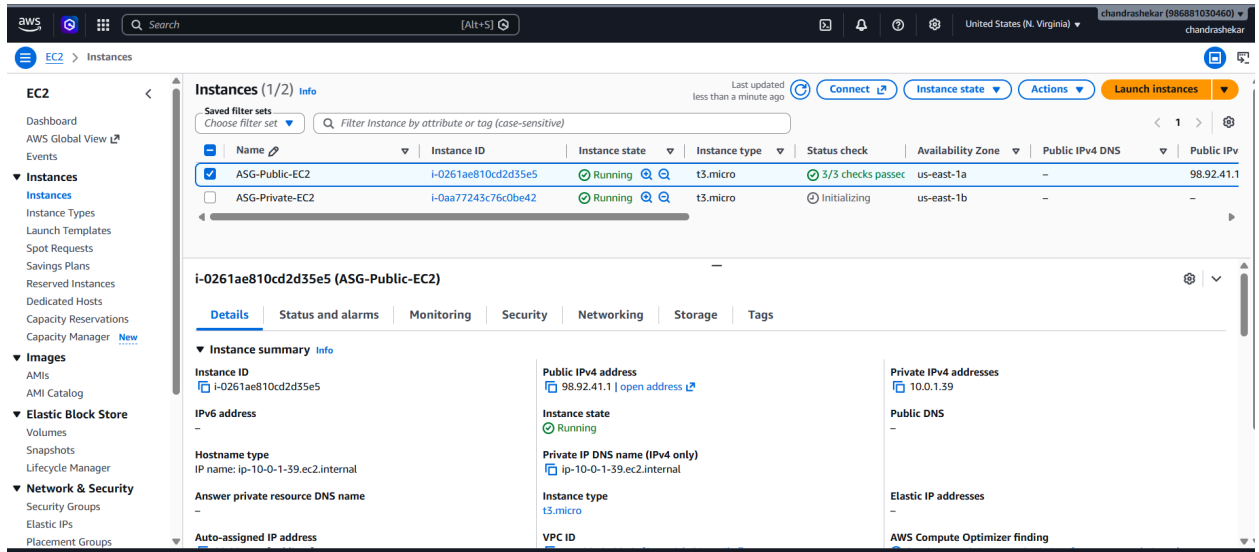
- Name: **ASG-Private-EC2**
- AMI: same Ubuntu AMI
- Instance type: **t3.micro**
- VPC: **ASG-VPC**
- Subnet: **ASG-Private-Subnet**
- **Auto-assign public IP: Disable**
- Key pair: same key pair

For the security group, allow **HTTP 80** for now; we'll configure access appropriately when we set up the load balancer.

The most important check after launching is:

Public EC2: private IP **10.0.1.x** + public IP 

Private EC2: private IP **10.0.2.x** + **NO public IP** 



7. Deploy Apache server on both EC2 instances with a sample index.html file.

First we'll configure **Public EC2** only.

Select **ASG-Public-EC2** → **Connect** → **EC2 Instance Connect** → **Connect**.

Then run:

```
sudo apt update
sudo apt install apache2 -y

sudo systemctl enable apache2
sudo systemctl start apache2
sudo systemctl status apache2
```

Then create your sample page:

```
echo "<h1>Welcome to ASG Public Apache Server</h1>" | sudo tee
/var/www/html/index.html
```

Test inside the server:

```
curl http://localhost
```

You should get:

```
<h1>Welcome to ASG Public Apache Server</h1>
```

Then test from your browser using:

<http://98.92.41.1>

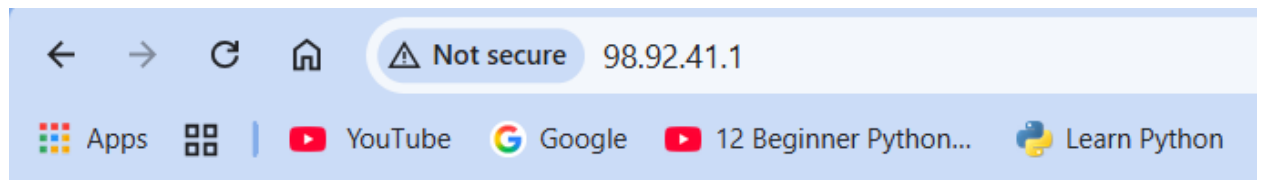
provided port **80** is allowed in the Public EC2 security group

```
ubuntu@ip-10-0-1-39: ~
Enabling module env.
Enabling module mime.
Enabling module negotiation.
Enabling module setenvif.
Enabling module filter.
Enabling module deflate.
Enabling module status.
Enabling module reqtimeout.
Enabling conf charset.
Enabling conf localized-error-pages.
Enabling conf other-vhosts-access-log.
Enabling conf security.
Enabling conf serve-cgi-bin.
Enabling site 000-default.
Created symlink /etc/systemd/system/multi-user.target.wants/apache2.service' -> '/usr/lib/systemd/system/apache2.service'.
Created symlink /etc/systemd/system/multi-user.target.wants/apache-htcacheclean.service' -> '/usr/lib/systemd/system/apache-htcacheclean.service'.
Processing triggers for ufw (0.36-2-9build1) ...
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-23 07:02:41 UTC; 7s ago
     Invocation: 4fd5b15298941d3ac3d7c63704c380e
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 2092 (apache2)
      Status: "Processing requests..."
        Tasks: 55 (limit: 627)
       Memory: 5.6M (peak: 5.8M)
          CPU: 87ms
    CGroup: /system.slice/apache2.service
            └─2092 /usr/sbin/apache2 -k start -DFOREGROUND
              └─2108 /usr/sbin/apache2 -k start -DFOREGROUND
                └─2109 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 23 07:02:41 ip-10-0-1-39 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 23 07:02:41 ip-10-0-1-39 apachectl[2092]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.1.39. Set the 'ServerName' d
Sep 23 07:02:41 ip-10-0-1-39 systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-18/18 (END)
[1]~ Stopped
ubuntu@ip-10-0-1-39:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-23 07:02:41 UTC; 7s ago
     Invocation: 4fd5b15298941d3ac3d7c63704c380e
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 2092 (apache2)
      Status: "Processing requests..."
        Tasks: 55 (limit: 627)
       Memory: 5.6M (peak: 5.8M)
          CPU: 87ms
    CGroup: /system.slice/apache2.service
            └─2092 /usr/sbin/apache2 -k start -DFOREGROUND
              └─2108 /usr/sbin/apache2 -k start -DFOREGROUND
                └─2109 /usr/sbin/apache2 -k start -DFOREGROUND

<h1>Welcome to ASG Public Apache Server</h1>" | sudo tee /var/www/html/index.html
ubuntu@ip-10-0-1-39:~$ curl http://localhost
<h1>Welcome to ASG Public Apache Server</h1>
ubuntu@ip-10-0-1-39:~$
```



On the **SSH** row only:

1. Click **Delete** on the right side of the SSH rule.
2. Click **Add rule**.

3. Choose:
 - **Type:** SSH
 - **Port:** 22
 - **Source:** Custom
4. In the source search box, start typing/selecting:

sg-0e5cc73cf2449e3f3
5. **Select it from the AWS dropdown** when it appears. Don't just type the ID and leave it as text.
6. Click **Save rules**.

Keep your HTTP rule unchanged:

HTTP 80 0.0.0.0/0

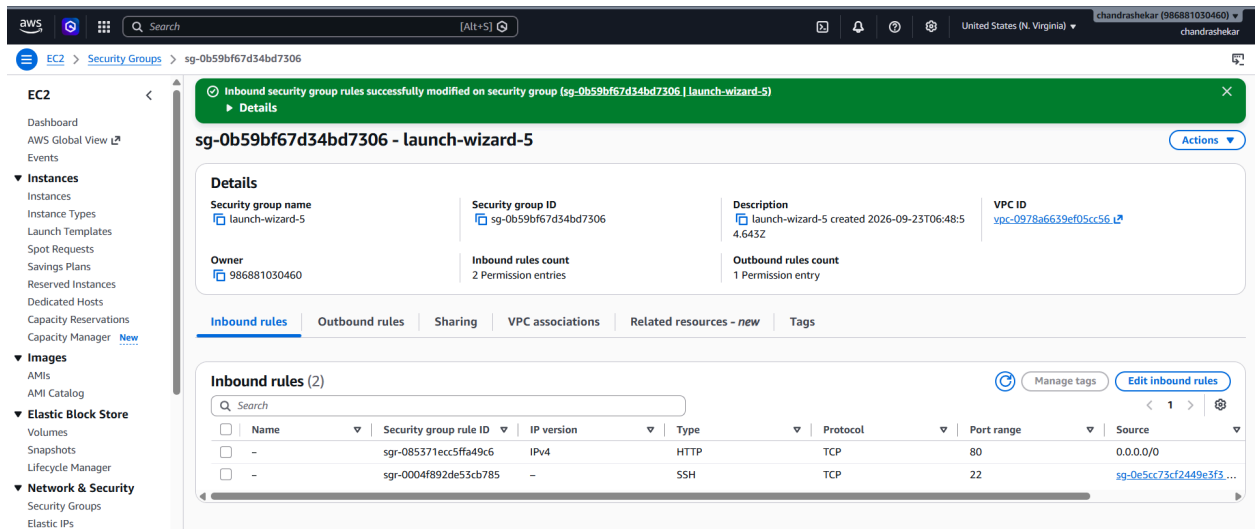
Final result should be:

HTTP TCP 80 0.0.0.0/0

SSH TCP 22 sg-0e5cc73cf2449e3f3

Important: this assumes sg-0e5cc73cf2449e3f3 is actually the security group attached to your **ASG-Public-EC2**.

Delete the current SSH row → **Add rule** → select the SG, and it should work.



Best way: SSH Agent Forwarding

Do this from your **Windows Git Bash**, not from ubuntu@ip-10-0-1-39.

First exit the public EC2:

exit

Now you should be back at something like:

```
HP@DESKTOP-1GQO65F MINGW64 ~  
$
```

Find your key. For example, if it is in Downloads:

```
ls ~/Downloads/*.pem
```

Suppose your key is:

```
~/Downloads/ASG-Key.pem
```

Then run:

```
eval "$(ssh-agent -s)"  
ssh-add ~/Downloads/ASG-Key.pem  
ssh-add -l
```

You should see your key listed.

Now connect from Windows → Public EC2 **with agent forwarding**:

```
ssh -A ubuntu@98.92.41.1
```

Once you're inside the public EC2:

```
ssh ubuntu@10.0.2.221
```

The path will be:

```
Your Windows PC  
|  
| .pem key + SSH agent  
▼  
Public EC2  
98.92.41.1  
|  
| SSH port 22  
▼  
Private EC2
```

10.0.2.221

```
Connection to 98.92.41.1 closed.

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ ls ~/Downloads/*.pem
/c/Users/HP/Downloads/my_server.pem /c/Users/HP/Downloads/west.pem

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ eval "$(ssh-agent -s)"
Agent pid 714

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ ssh-add ~/Downloads/ASG-Key.pem
/c/Users/HP/Downloads/ASG-Key.pem: No such file or directory

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ ssh-add ~/Downloads/my_server.pem
Identity added: /c/Users/HP/Downloads/my_server.pem (/c/Users/HP/Downloads/my_server.pem)

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ ssh-add -l
2048 SHA256:cvKRTifLQMvPa9Bdd3WAnv3XlspbqKRu9Awp4mAM3g /c/Users/HP/Downloads/my_server.pem (RSA)

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)
$ ssh -A ubuntu@98.92.41.1
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Sep 23 07:33:09 UTC 2026

System load: 0.0           Temperature: -273.1 C
Usage of /: 34.6% of 6.61GB Processes: 127
Memory usage: 30%         Users logged in: 0
Swap usage: 0%            IPv4 address for ens5: 10.0.1.39

Expanded Security Maintenance for Applications is not enabled.

186 updates can be applied immediately.
136 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Sep 23 07:26:25 2026 from 103.160.27.134
ubuntu@ip-10-0-1-39:~$ |
```

28°



```
ubuntu@ip-10-0-2-221:~$ sudo systemctl enable apache2
Enabling module env.
Enabling module mime.
Enabling module negotiation.
Enabling module setenvif.
Enabling module filter.
Enabling module deflate.
Enabling module status.
Enabling module reqtimeout.
Enabling conf charset.
Enabling conf localized-error-pages.
Enabling conf other-vhosts-access-log.
Enabling conf security.
Enabling conf serve-cgi-bin.
Enabling site 000-default.
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' → '/usr/lib/systemd/system/apache2.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/apache-htcacheclean.service' → '/usr/lib/systemd/system/apache-htcacheclean.service'.
Processing triggers for ufw (0.36.2-9build1) ...
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-23 07:36:55 UTC; 8s ago
     Invocation: c6a6b49cfe14973b5e31c2605e0d6df
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 2178 (apache2)
      Status: "Processing requests..."
        Tasks: 55 (limit: 627)
       Memory: 5.6M (peak: 5.8M)
          CPU: 114ms
    CGroup: /system.slice/apache2.service
            └─2178 /usr/sbin/apache2 -k start -DFOREGROUND
              └─2198 /usr/sbin/apache2 -k start -DFOREGROUND
                └─2199 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 23 07:36:55 ip-10-0-2-221 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 23 07:36:55 ip-10-0-2-221 apache2[2178]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.2.221. Set the 'ServerName' directive dynamically to determine the server's name.
Sep 23 07:36:55 ip-10-0-2-221 systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-18/18 (END)
[1]~$ sudo systemctl status apache2
ubuntu@ip-10-0-2-221:~$ echo "<h1>Welcome to ASG Private Apache Server</h1>" | sudo tee /var/www/html/index.html
curl http://localhost
<h1>Welcome to ASG Private Apache Server</h1>
<h1>Welcome to ASG Private Apache Server</h1>
ubuntu@ip-10-0-2-221:~$
```

So now you have both servers ready:

Instance	Subnet	Apa che
ASG-Publi c-EC2	Public 10.0.1.x	✓ Run ning
ASG-Priv ate-EC2	Private 10.0.2.2 21	✓ Run ning

Also, because you were able to install packages on the **private EC2**, your **Private Route Table** → **NAT Gateway** → **Internet** path is working. ✓

8. Create one application load balancer and attach it to both EC2 instances.

Go to:

EC2 → Load Balancing → Target Groups → Create target group

Use:

Target type: Instances

Target group name: ASG-Target-Group

Protocol: HTTP

Port: 80

VPC: ASG-VPC

Protocol version: HTTP1

Health check protocol: HTTP

Health check path: /

Then click **Next**.

On **Register targets**, select:

ASG-Public-EC2

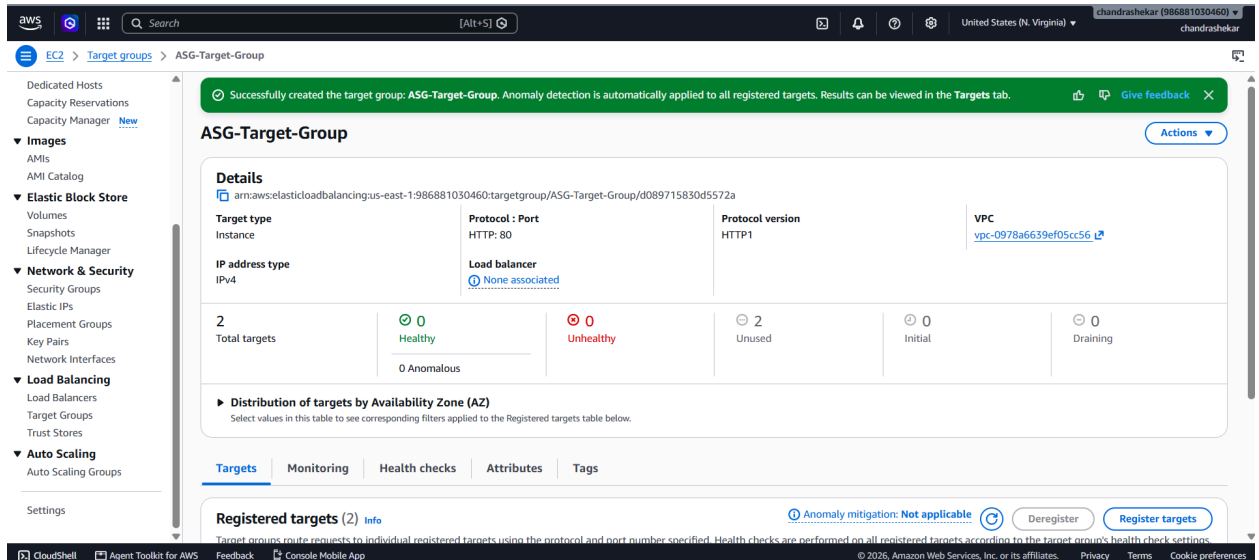
ASG-Private-EC2

Yes, this screen is correct. ☒ Both instances are appearing:

- ASG-Private-EC2 — 10.0.2.221 — us-east-1b
- ASG-Public-EC2 — 10.0.1.39 — us-east-1a
- Target port **80** — correct.

Now do exactly this:

1. Tick ☒ **both instances**.
2. Keep **Ports for selected instances = 80**.
3. Click **Include as pending below**.
4. Scroll down to **Review targets**. You should see both EC2 instances there.
5. Then continue to **Review and create → Create target group**.



An **Application Load Balancer** requires subnets in at least two **Availability Zones**. You currently have:

Public subnet → us-east-1a

Private subnet → us-east-1b

For an **internet-facing ALB**, don't use your private subnet as the second ALB subnet. Create **one additional public subnet** in another AZ, for example:

ASG-Public-Subnet-2

VPC: ASG-VPC

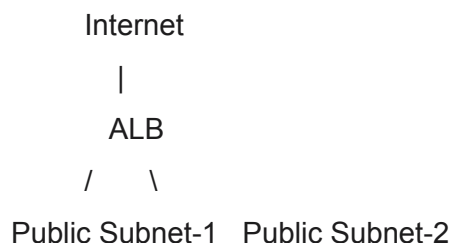
Availability Zone: us-east-1b

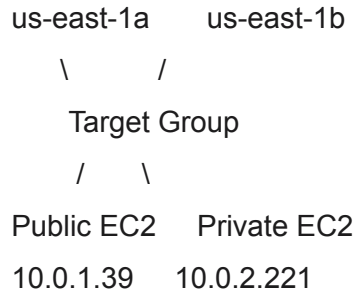
CIDR: 10.0.3.0/24

Then associate **ASG-Public-Subnet-2** with your existing **ASG-Public-RT**, which already has:

0.0.0.0/0 → Internet Gateway

After that, our design becomes:





That is the right setup for your task.

Next: Go to **VPC** → **Subnets** → **Create subnet**, select **ASG-VPC**, and create **ASG-Public-Subnet-2** with **10.0.3.0/24** in **us-east-1b**.

VPC → **Route tables** → click **ASG-Public-RT**

Then:

Subnet associations → **Edit subnet associations**

You should select **both**:

- ☒ ASG-Public-Subnet 10.0.1.0/24
- ☒ ASG-Public-Subnet-2 10.0.3.0/24

⚠ Do not select ASG-Private-Subnet. That must stay associated with **ASG-Private-RT** → **NAT Gateway**.

Click **Save associations**.

Go to **EC2** → **Load Balancing** → **Load Balancers** → **Create load balancer** → **Application Load Balancer**.

Use these settings:

Load balancer name: ASG-ALB

Scheme: Internet-facing

IP address type: IPv4

VPC: ASG-VPC

Mappings:

☒ us-east-1a → ASG-Public-Subnet

☒ us-east-1b → ASG-Public-Subnet-2

Listener:

HTTP : 80

Default action:

Forward to → ASG-Target-Group

For **Security Group**, we should create/use an ALB security group that allows:

HTTP | TCP | 80 | 0.0.0.0/0

Name: ASG-ALB

Scheme: Internet-facing

IP type: IPv4

VPC: ASG-VPC

AZ 1:

us-east-1a

ASG-Public-Subnet (10.0.1.0/24)

AZ 2:

us-east-1b

ASG-Public-Subnet-2 (10.0.3.0/24)

Security Group:

ASG-ALB-SG

Listener:

HTTP : 80

Default action:

Forward to → ASG-Target-Group

The screenshot shows the AWS Management Console interface for an ASG-ALB. The top navigation bar includes the AWS logo, a search bar, and the user's profile. The left sidebar contains a navigation menu with categories like Dedicated Hosts, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area displays the details of the ASG-ALB, including a success message, a notification about a new Capacity Utilization metric, and a table of details. The details table includes fields for Load balancer type, Status, VPC, Load balancer IP address type, Scheme, Hosted zone, Availability Zones, Load balancer ARN, and DNS name info. The bottom of the console shows a footer with links to CloudShell, Agent Toolkit for AWS, Feedback, and Console Mobile App, along with copyright information and privacy links.

ASG-ALB			
Details			
Load balancer type Application	Status Provisioning	VPC vpc-0978a6639ef05cc56	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z355XD0TRQ7X7K	Availability Zones subnet-01a6b737b03d4bb19 (us-east-1b) subnet-0cce9916d4412cf93 (us-east-1a)	Date created September 23, 2026, 14:59 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:986881030460:loadbalancer/app/ASG-ALB/cee1dece209a6b77		DNS name info ASG-ALB-1180530727.us-east-1.elb.amazonaws.com (A Record)	

EC2 → Target Groups → **ASG-Target-Group** → Targets

We need to verify your two instances show:

ASG-Public-EC2 Healthy

ASG-Private-EC2 Healthy

If both become **Healthy**, then copy the **DNS name** shown on this ALB page and open it in the browser. Refresh several times. Since your two Apache servers have different pages, you should see responses such as:

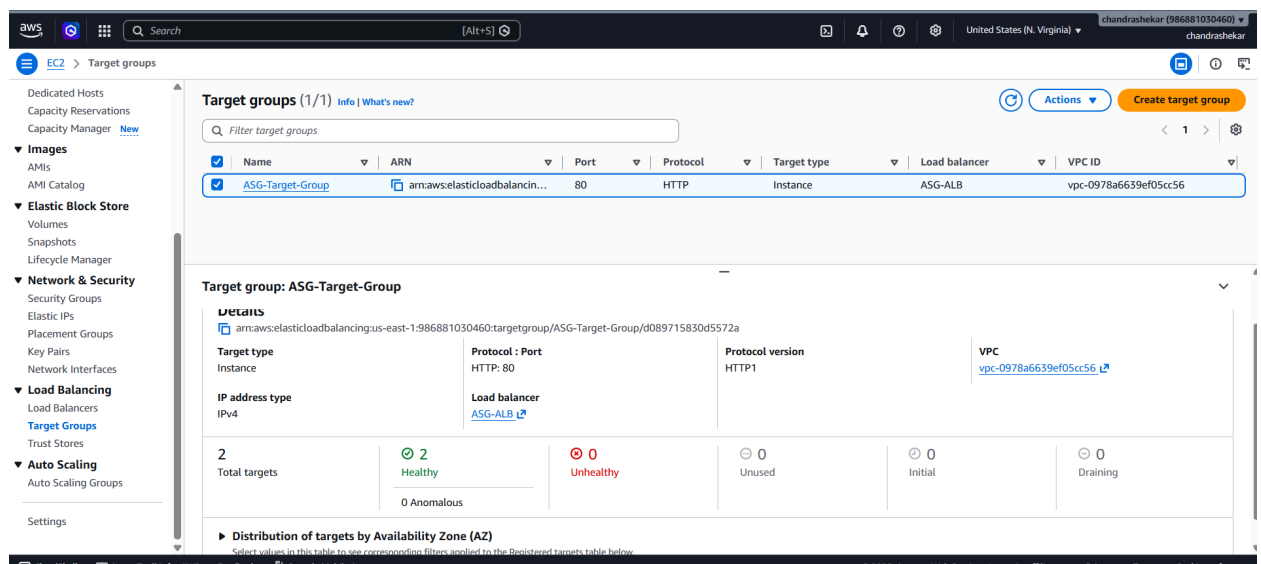
Welcome to ASG Public Apache Server

and

Welcome to ASG Private Apache Server

That will prove:

Internet → ALB → Target Group → Public/Private EC2 → Apache is working.



9.Store application load balancer logs in S3.

Step 1 — Create an S3 bucket

Go to **AWS Console** → **S3** → **Buckets** → **Create bucket**.

Use:

AWS Region: US East (N. Virginia) us-east-1

Bucket name:

asg-alb-logs-986881030460

Bucket names must be globally unique, so if AWS says the name already exists, use something like:

asg-alb-logs-986881030460-2026

For now, keep:

Object Ownership:

ACLs disabled

Block Public Access:

Keep ALL options checked 

Bucket Versioning:

Disable

Default encryption:

SSE-S3

Don't make this bucket public. ALB logging uses S3 permissions; public access isn't required.

Click **Create bucket**.

What we're building

Users



ASG-ALB



Apache EC2s

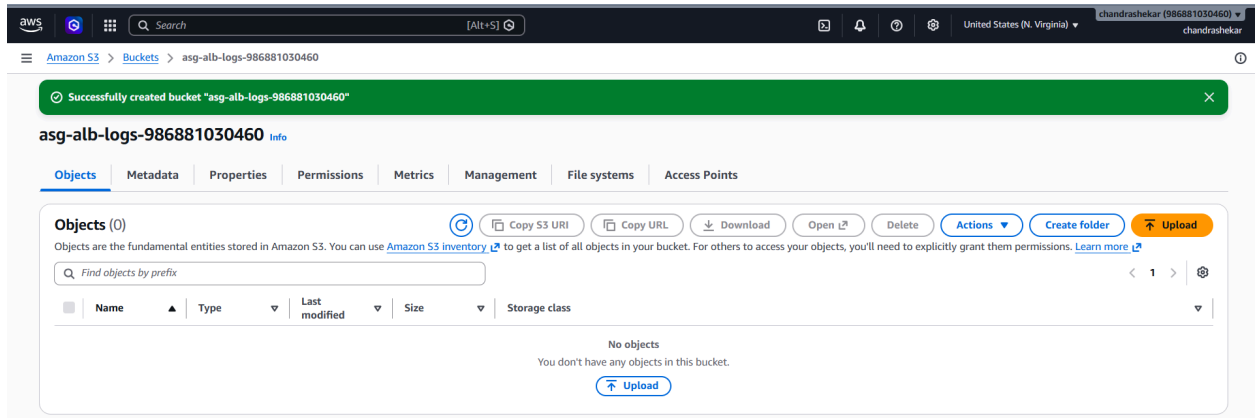
ASG-ALB



└─ Access Logs

↓
S3 Bucket

The logs will later contain information about requests reaching your ALB, such as request time, client address, target, HTTP request, response codes, and processing information.



Step 2 — Add the bucket policy

Now click:

Permissions → **Bucket policy** → **Edit**

Paste this policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "logdelivery.elasticloadbalancing.amazonaws.com"
      },
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::asg-alb-logs-986881030460/AWSLogs/986881030460/*"
    }
  ]
}
```

Then click **Save changes**.

Keep **Block Public Access ON**. Don't change it.

After saving the policy, we'll go to:

EC2 → Load Balancers → ASG-ALB → Attributes → Access logs

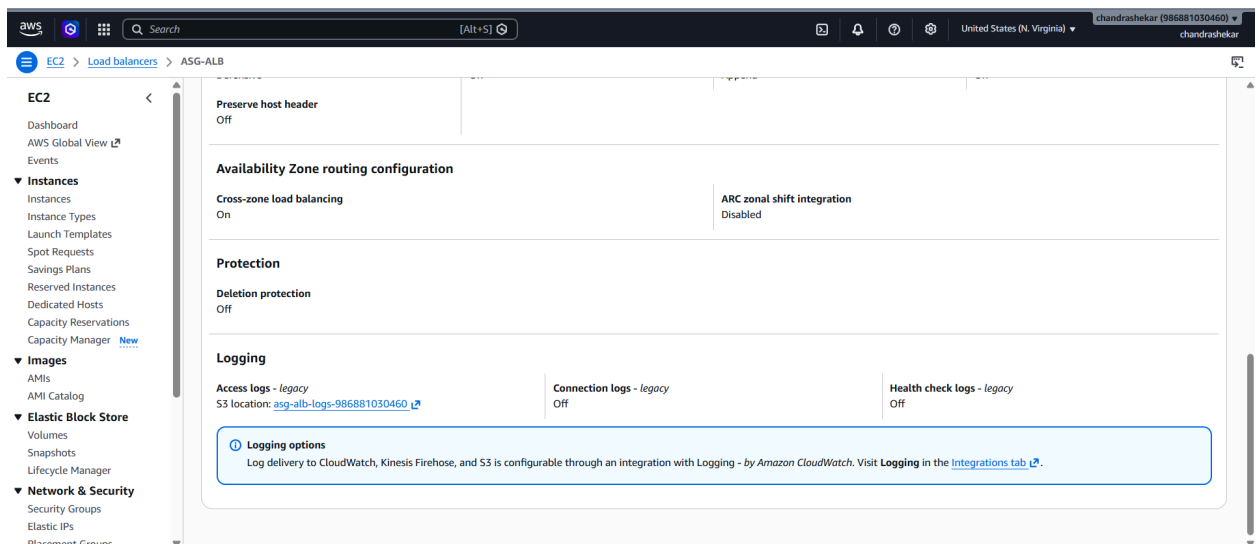
and connect this S3 bucket.

Your AWS console has the newer logging options. Under **Logging**, you can see:

- **Access logs – legacy**
- Connection logs – legacy
- Health check logs – legacy
- And the newer **Logging – by Amazon CloudWatch** integration.

For our current task of storing **ALB access logs in the S3 bucket you created**, do this:

1. ☒ Tick **Access logs – legacy**.
2. After you tick it, additional fields should appear underneath.
3. For **S3 bucket**, enter/select:
`asg-alb-logs-986881030460`
4. If it asks for a prefix, use something simple like:
`alb-logs`
5. Then click **Save changes**.



Access logs – legacy → S3 location: **asg-alb-logs-986881030460**.

Now generate some traffic through the **ALB DNS name**—open it in your browser and refresh several times. Then wait around **5–10 minutes** and go to **S3** →

asg-alb-logs-986881030460 → **Objects**. You should start seeing the ALB log path/files, typically under an **AWSLogs/.../elasticloadbalancing/...** structure.

So this part of your architecture is now:

User → **Internet-facing ALB** → **Target Group** → **EC2 Apache servers**

ALB → **Access Logs** → **S3 bucket**

Go to:

AWS Console → **EC2** → **Launch Templates** → **Create launch template**

Use approximately these settings:

Setting	Value
Launch template name	ASG-Web-LT
AMI	Ubuntu Server
Instance type	t3.micro
Key pair	Your existing key pair
VPC	ASG-VPC
Security Group	Web/EC2 security group

Storage **Default**

User data **Install Apache
automatically**

Put this in Advanced details → User data:

```
#!/bin/bash
```

```
apt update -y
```

```
apt install apache2 -y
```

```
systemctl enable apache2
```

```
systemctl start apache2
```

```
INSTANCE_ID=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
```

```
echo "<h1>Welcome to Auto Scaling Web Server</h1>
```

```
<h2>Instance: $INSTANCE_ID</h2>" > /var/www/html/index.html
```

Then click Create launch template.

After that we'll create:

Launch Template → Auto Scaling Group → Private subnets → Attach your existing
ASG-Target-Group → Min 2 / Desired 2 / Max 4 → CPU scaling policy

That will give you the important real-time flow:

Internet → ALB → Target Group → Auto Scaling EC2 instances

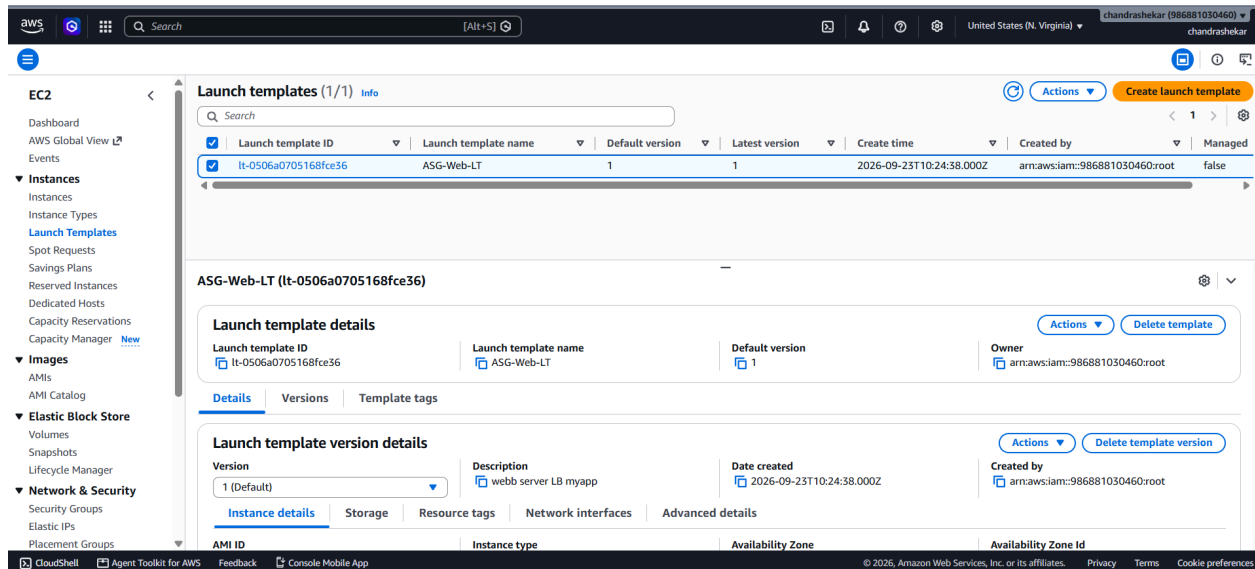
When load increases:

ASG → launches more EC2

When load decreases:

ASG → terminates unnecessary EC2

Go to EC2 → Launch Templates → Create launch template



Go to the left menu:

EC2 → Auto Scaling → Auto Scaling Groups → Create Auto Scaling group

On the first page:

Name: ASG-Web-ASG

Launch template: ASG-Web-LT

Version: Default (1)

Click Next.

Network page — important

When you reach Choose instance launch options, select:

VPC: ASG-VPC

For the ASG instances, I recommend using private subnets, not the two public subnets used by the ALB.

Right now you have one private subnet:

ASG-Private-Subnet → 10.0.2.0/24

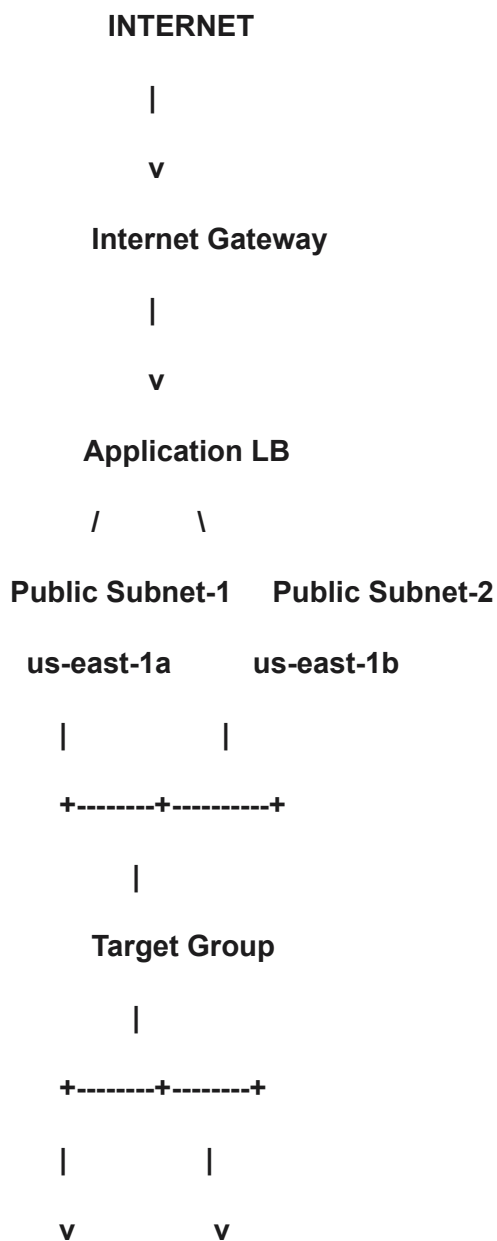
For proper Multi-AZ Auto Scaling, create another private subnet in the second AZ, for example:

ASG-Private-Subnet-2 → 10.0.4.0/24

Then associate that subnet with your private route table, which has:

0.0.0.0/0 → NAT Gateway

Your finished layout will be:



Private Subnet-1 Private Subnet-2

10.0.2.0/24 10.0.4.0/24

EC2 EC2

\ /

\-- Auto Scaling --/

|

NAT Gateway

|

Internet

(outbound traffic only)

So before finishing the ASG, create **ASG-Private-Subnet-2** in the other AZ. That gives you a much better real-world architecture.

Go to VPC → Subnets → Create subnet

Click Create subnet.

Enter:

Setting	Value
VPC ID	ASG-VPC
Subnet name	ASG-Private-Subnet-2
Availability Zone	Choose an AZ different from your first private subnet

IPv4 VPC CIDR block **10.0.0.0/16**

IPv4 subnet CIDR block **10.0.4.0/24**

From your earlier setup, your first private EC2 was in us-east-1b. So choose us-east-1a for **ASG-Private-Subnet-2**.

Then click Create subnet.

2. Associate it with your private route table

After creation:

VPC → Route tables → **ASG-Private-RT** → Subnet associations → Edit subnet associations

You currently have:

ASG-Private-Subnet 10.0.2.0/24

Tick the new:

ASG-Private-Subnet-2 10.0.4.0/24

Click Save associations.

3. Verify the private route

Open:

ASG-Private-RT → Routes

You should already have something like:

Destination	Target
-------------	--------

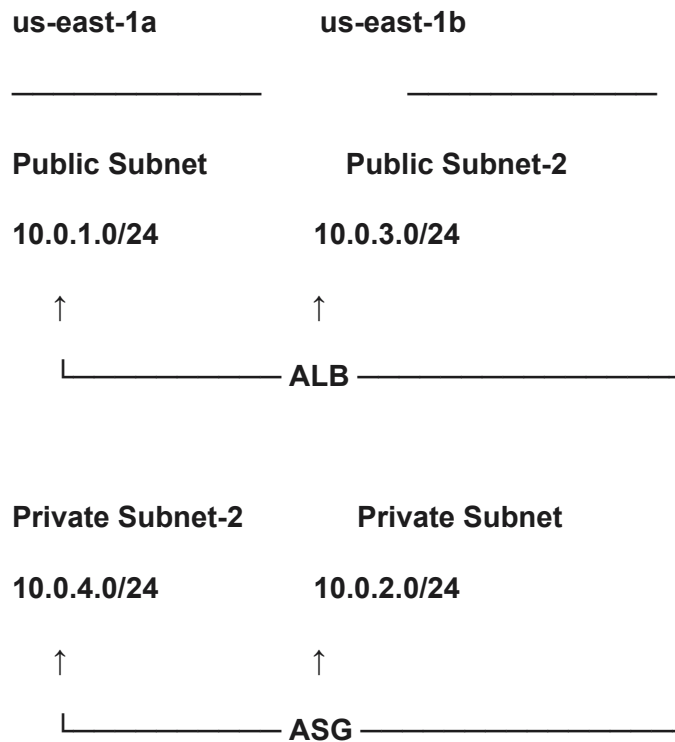
10.0.0.0/16	local
-------------	-------

0.0.0.0/0	NAT Gateway
-----------	-------------

Don't create another NAT Gateway for this lab. Your existing NAT route can be used.

Then your subnet design becomes

ASG-VPC 10.0.0.0/16



Once **ASG-Private-Subnet-2** is created and associated with **ASG-Private-RT**, we can go straight back to Auto Scaling Groups → Create Auto Scaling group.

Step 2 – Choose instance launch options screen. 🍌

I can see **ASG-VPC** is selected correctly, but currently you have selected only one private subnet:

ASG-Private-Subnet → 10.0.2.0/24 → us-east-1b

You should select both private subnets here. Open Select Availability Zones and subnets and select:

- **ASG-Private-Subnet → 10.0.2.0/24 → us-east-1b**
- **ASG-Private-Subnet-2 → 10.0.4.0/24 → the other AZ (for example us-east-1a)**

After selecting them, you should see two blue subnet boxes, not just one.

For Availability Zone distribution, keep Balanced best effort selected. 

Then click Next to reach Step 3 – Integrate with other services.

Under Load balancing, change the selection from No load balancer to:

- Attach to an existing load balancer

After selecting it, AWS will show additional options underneath. Since you already created:

ALB: **ASG-ALB**

Target Group: **ASG-Target-Group**

choose the option for an existing load balancer target group, then select **ASG-Target-Group**.

Do not choose Attach to a new load balancer because your ALB already exists.

Keep VPC Lattice = No VPC Lattice service. 

I can see you have selected Attach to an existing load balancer → Choose from your load balancer target groups → **ASG-Target-Group** | **HTTP**. It also correctly shows Application Load Balancer: **ASG-ALB**. Keep No VPC Lattice service selected.

Now scroll down on this same page. If you see Health checks, enable Turn on Elastic Load Balancing health checks if available. That lets the ASG use the load balancer's target health when deciding whether an instance should be replaced.

Then click Next to go to Step 4 — Configure group size and scaling.

For our lab, when you reach that screen, we'll set approximately:

Desired capacity = 2, Minimum = 2, Maximum = 4, then configure the scaling policy.

What these mean

Desired = 2: AWS initially maintains 2 EC2 instances. Since you selected two private subnets in different AZs, Auto Scaling can distribute the instances across them.

Minimum = 2: Even when traffic is very low, ASG should not go below 2 instances.

Maximum = 4: When load increases, ASG is allowed to scale out up to 4 instances.

So conceptually:

ALB → Target Group → ASG → EC2 instances in private subnets

Normal load → 2 EC2

High load → 3 → 4 EC2

Load decreases → 4 → 3 → 2 EC2

ASG-Web-ASG created successfully

Error: Failed to create 1 Scaling policy. EC2 Auto Scaling doesn't have permission to complete your request because the service-linked role isn't available yet. You might just need to wait a few minutes before trying again. It can take from a few seconds to a number of minutes for new roles to propagate.

Auto Scaling groups (1/1) Info

Search your Auto Scaling groups

Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired capacity	Min	Max	Availability Zone
ASG-Web-ASG	At desired capacity	ASG-Web-LT Version Default	2	2/2 Healthy	2	2	4	2 Availability Zone

Auto Scaling group: ASG-Web-ASG

So the ASG successfully launched the two EC2 instances. The red message only means the **CPU target-tracking scaling policy failed to create earlier** because the AWS service-linked role was still propagating.

Next: create the scaling policy again

Click **ASG-Web-ASG** → find the **Automatic scaling** tab → **Create dynamic scaling policy**.

Use:

Policy type: Target tracking scaling

Policy name: CPU-Target-Tracking

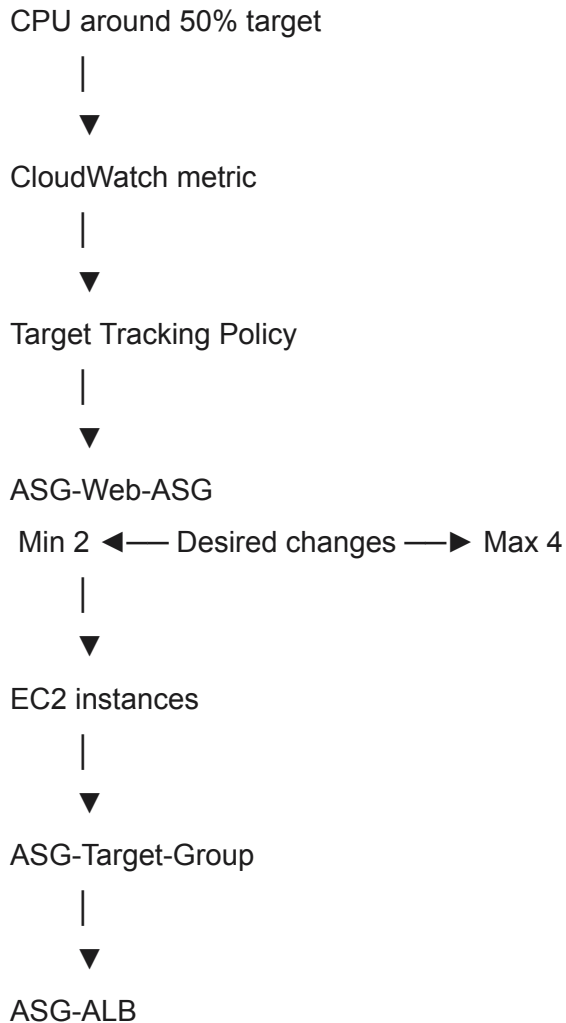
Metric type: Average CPU utilization

Target value: 50

Instance warmup: 300 seconds

Keep **Disable scale in** unchecked.

This gives you:



Once we create that policy, the important next test is **ALB → Target Group → 2 healthy targets → open ALB DNS name in browser**. Then we can actually generate load and watch Auto Scaling go from **2 → 3/4 instances**.

EC2 > Target groups

Dedicated Hosts

Capacity Reservations

Capacity Manager New

▼ Images

AMIs

AMI Catalog

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

▼ Load Balancing

Load Balancers

Target Groups

Trust Stores

▼ Auto Scaling

Auto Scaling Groups

Settings

Target groups (1/1) Info What's new?

Filter target groups

☒	Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
☒	ASG-Target-Group	arn:aws:elasticloadbalancing...	80	HTTP	Instance	ASG-ALB	vpc-0978a6639ef05cc56

Target group: ASG-Target-Group

IP address type

IPv4

Load balancer

ASG-ALB

Target access analysis

Access issue detected

4

2

0

2

0

0

Total targets

Healthy

Unhealthy

Unused

Initial

Draining

0 Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

EC2 > Target groups > ASG-Target-Group

EC2

Dashboard

AWS Global View

Events

▼ Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager New

▼ Images

AMIs

AMI Catalog

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Zone

Total targets

Healthy

Unhealthy

Unused

Initial

us-east-1b (use1-az2)	3	2	0	0	0
us-east-1a (use1-az1)	1	1	0	0	0

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (4) Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Admini...	Overri...	Launch...
☐	i-05c11224e53674279	ASG-web-server	80	us-east-1b (us...	Healthy	-	No override.	No overri...	Septembe...
☐	i-0e8174a17ca9f0f15	ASG-web-server	80	us-east-1b (us...	Draining	Target deregistration i...	No override.	No overri...	Septembe...
☐	i-0aa77243c76c0be42	ASG-Private-EC2	80	us-east-1b (us...	Healthy	-	No override.	No overri...	Septembe...
☐	i-0261ae810cd2d35e5	ASG-Public-EC2	80	us-east-1a (us...	Healthy	-	No override.	No overri...	Septembe...

24 of 24 - Clipboard

Item not Collected: Format not supported by Office Clipboard

CloudShell

Agent Toolkit for AWS

Feedback

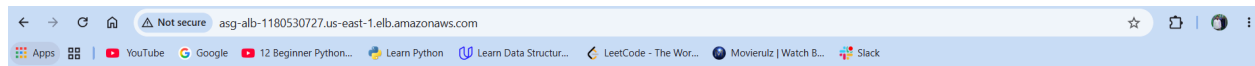
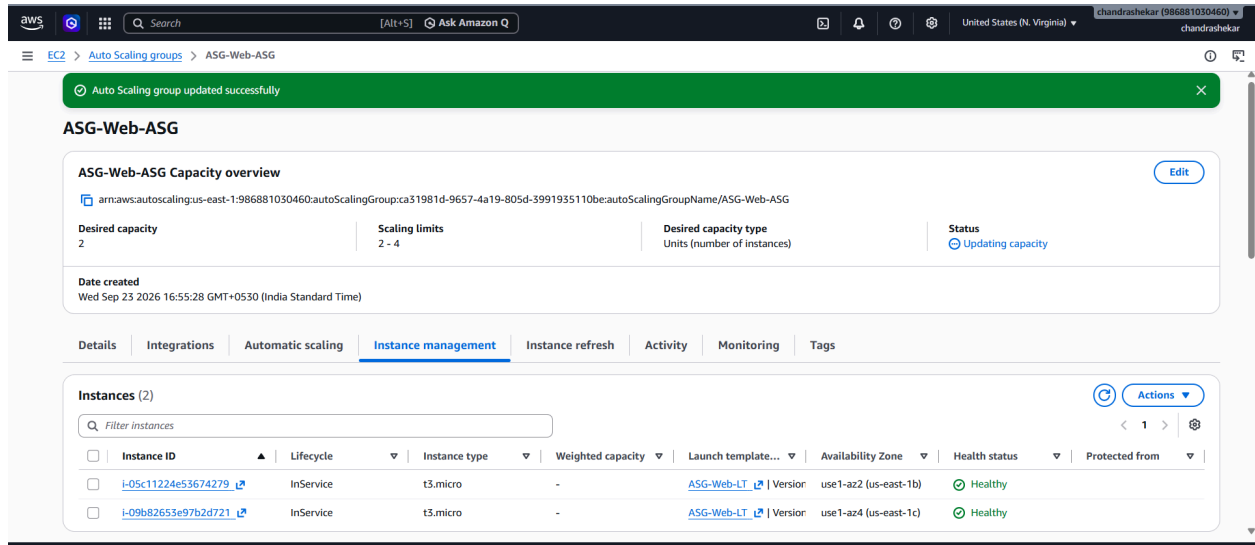
Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences



Welcome to My AWS Auto Scaling Application

Apache Web Server is Running

This EC2 instance was created automatically by Auto Scaling Group.

I created a Launch Template containing the EC2 configuration and user data to install Apache. I created an Auto Scaling Group using that Launch Template and distributed the instances across multiple Availability Zones. I attached the ASG to a Target Group and connected the Target Group to an internet-facing Application Load Balancer. When I access the ALB DNS name, the load balancer forwards the HTTP request to a healthy EC2 instance, and my application is displayed.”

10.Store the VPC flow logs in a CloudWatch log group.

Go to:

AWS Console → VPC → Your VPCs → select ASG-VPC

Then look at the tabs at the bottom. You should find **Flow logs**.

Click **Flow logs → Create flow log**.

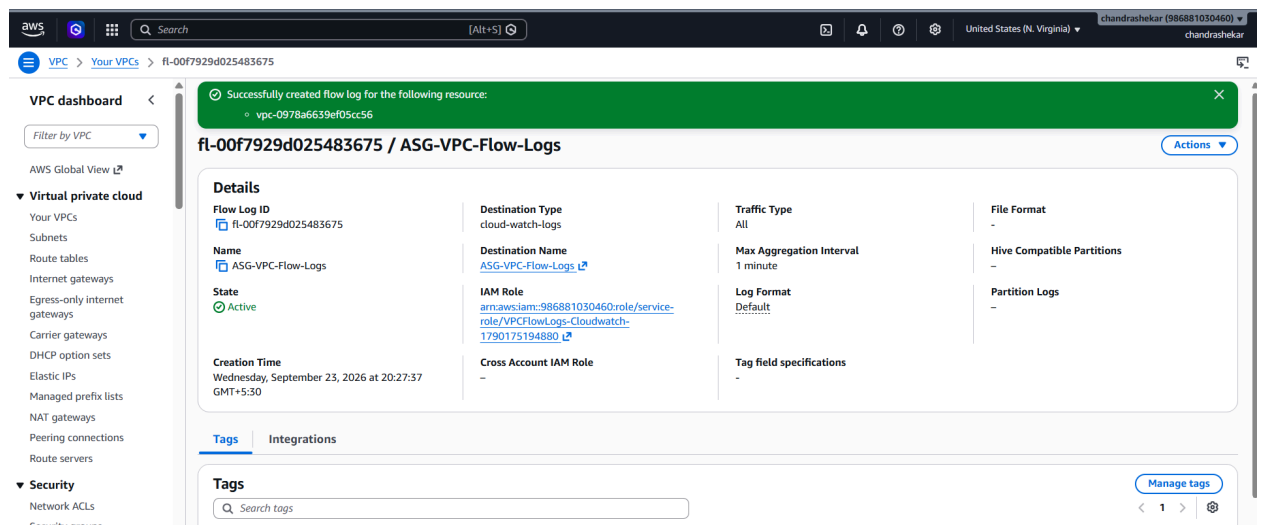
Name: ASG-VPC-Flow-Logs

Filter: All ☒

Maximum aggregation interval: 1 minute — better for this monitoring practice

- **Destination log group:** we'll create/select a CloudWatch log group such as ASG-VPC-Flow-Logs
- **Service access:** Create and use a new service role
- **Log record format:** AWS default format ☒
- **Additional metadata:** leave unchecked
- **EC2 tag fields:** leave unchecked

After you click **Create**



- **Flow Log:** ASG-VPC-Flow-Logs
- **State:** ☒ Active
- **Destination:** CloudWatch Logs
- **Traffic type:** All
- **Aggregation interval:** 1 minute
- **IAM role:** Created and attached
- **Log format:** Default

So **Store the VPC Flow Logs in a CloudWatch Log Group — DONE** ☒

11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Go to **AWS Console search** → **CloudWatch** → **Dashboards** → **Create dashboard**.

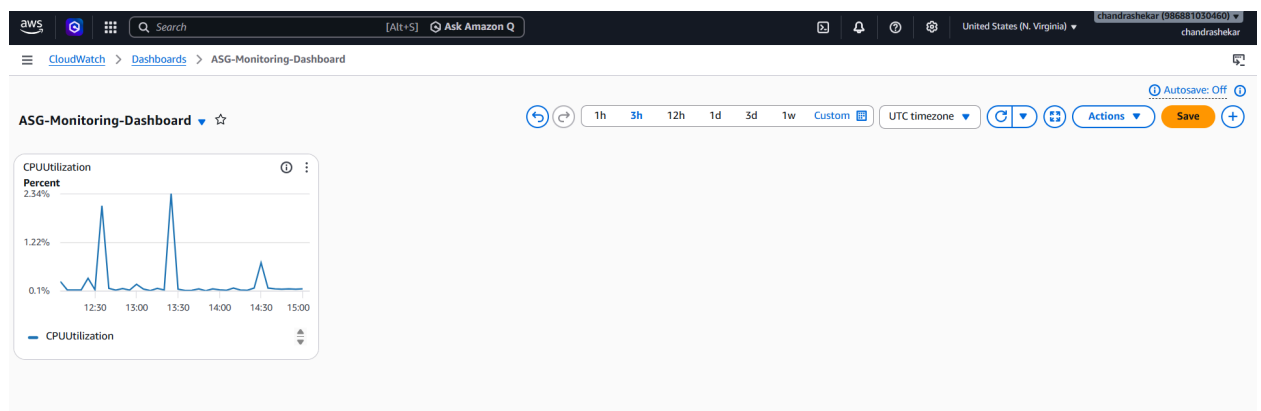
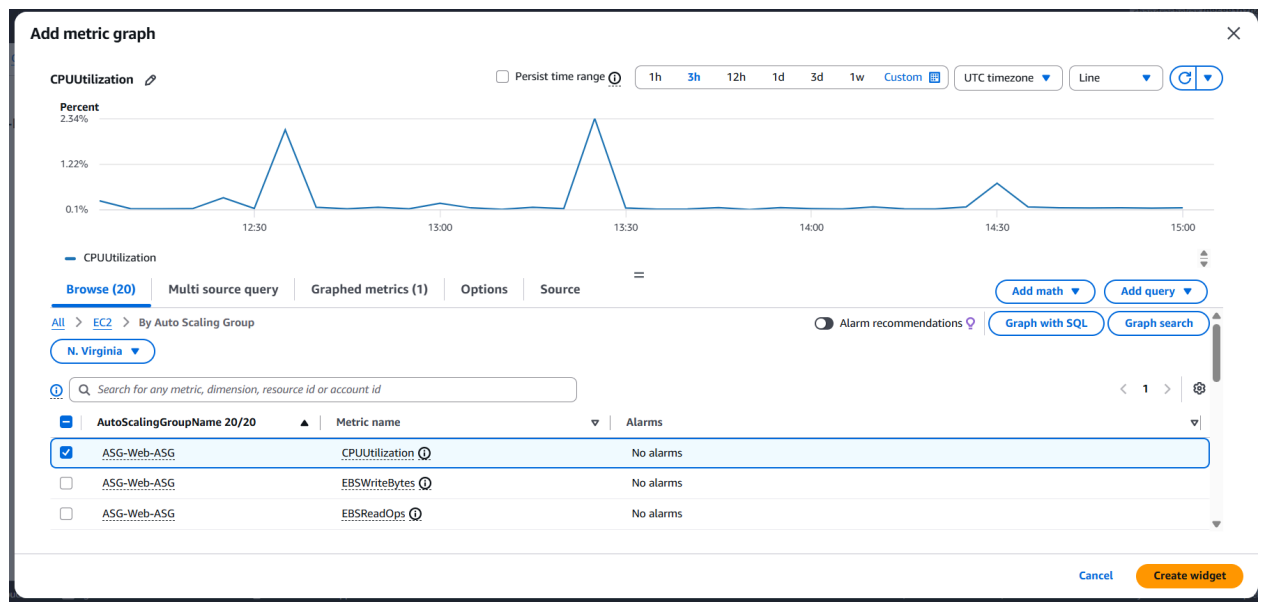
After clicking it, AWS should open the dashboard and ask you to **Add widget** / choose a widget type.

CloudWatch → Metrics → Metrics Console → Line

Now click the orange **Next** button at the bottom-right.

On the next screen, we need to select:

EC2 → By Auto Scaling Group → CPUUtilization



CPU monitoring is now done. 

Go to:

AWS Console → IAM → Roles → Create role

Then select:

Trusted entity type: **AWS service**

Use case: **EC2**

Then click **Next**.

On the permissions page, search for:

CloudWatchAgentServerPolicy

Select **CloudWatchAgentServerPolicy** and continue.

For the role name use:

EC2-CloudWatch-Monitoring-Role

Then click **Create role**.

Why are we doing this?

The flow will be:

Apache EC2 → check httpd/apache2 → publish custom metric → CloudWatch → Dashboard

Then your dashboard can show something like:

ApacheServiceStatus = 1 → Apache running 

ApacheServiceStatus = 0 → Apache stopped 

Go to EC2 → Launch Templates → select ASG-Web-LT → Actions → Modify template (Create new version).

On the Modify template page, scroll down and expand Advanced details. Find IAM instance profile and select the instance profile corresponding to:

EC2-CloudWatch-Monitoring-Role

Do not change the AMI, instance type, security group, or other settings.

Then scroll to the bottom and click **Create template version**

Go to **EC2** → **Auto Scaling Groups** → **ASG-Web-ASG** → **Details**.

Then scroll to the **Launch template** section and click **Edit**.

On the **Edit** page:

1. Keep **Launch template** = **ASG-Web-LT**
2. Under **Version**, choose the new/latest version you just created (probably **Version 3**).
3. Scroll down and click **Update**.

EC2 > Auto Scaling groups > ASG-Web-ASG

Auto Scaling group updated successfully

ASG-Web-ASG

ASG-Web-ASG Capacity overview [Edit](#)

arn:aws:autoscaling:us-east-1:986881030460:autoScalingGroup:ca31981d-9657-4a19-805d-3991935110be:autoScalingGroupName/ASG-Web-ASG

Desired capacity 2	Scaling limits 2 - 4	Desired capacity type Units (number of instances)	Status At desired capacity
------------------------------	--------------------------------	---	--------------------------------------

Date created
Wed Sep 23 2026 16:55:28 GMT+0530 (India Standard Time)

[Details](#) [Integrations](#) [Automatic scaling](#) [Instance management](#) [Instance refresh](#) [Activity](#) [Monitoring](#) [Tags](#)

Launch template

Launch template [Edit](#)

lt-0506a0705168fce36
ASG-Web-LT

AMI ID
ami-0b6d9d5d33ba97d99

Instance type
t3.micro

Owner
arn:aws:iam::986881030460:root

Version
3

Security groups
-

Security group IDs
sg-00d234feb6afb75a3

Create time
Wed Sep 23 2026 20:54:54 GMT+0530 (India Standard Time)

12.If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

Objective

Configure an EC2 Auto Scaling Group so that when the average CPU utilization increases beyond the configured target of **70%**, Auto Scaling can increase the number of EC2 instances to handle the workload.

Steps Performed

1. **Opened the Auto Scaling Group**
 - AWS Console → **EC2**

- **Auto Scaling Groups**
- Selected **ASG-Web-ASG**
- 2. **Verified ASG Capacity**
 - Desired capacity: **2**
 - Minimum capacity: **2**
 - Maximum capacity: **4**
 - This allows the ASG to increase capacity up to 4 instances.
- 3. **Created a Dynamic Scaling Policy**
 - Opened **Automatic scaling**
 - Selected **Create dynamic scaling policy**
 - Policy type: **Target tracking scaling**
 - Metric type: **Average CPU utilization**
 - Target value: **70**
 - Instance warmup: **300 seconds**
- 4. **Created the Scaling Policy**
 - Clicked **Create**
 - AWS confirmed:
 “Dynamic scaling policy created or edited successfully.”
- 5. **Created an IAM Role for CloudWatch Monitoring**
 - Created role:
 EC2-CloudWatch-Monitoring-Role
 - Configured it for EC2/CloudWatch monitoring requirements.
- 6. **Updated the Launch Template**
 - Opened launch template:
 ASG-Web-LT
 - Created a new launch-template version.
 - Attached the required IAM instance profile/role.
 - Added/updated the required configuration for monitoring.
 - Latest launch template version became **Version 4**.
- 7. **Updated the Auto Scaling Group**
 - Updated **ASG-Web-ASG** to use the new launch-template version.
 - AWS displayed:
 “Auto Scaling group updated successfully.”
- 8. **Started Instance Refresh**
 - Opened **Instance refresh**
 - Selected **Replace instances**
 - Refresh strategy: **Rolling**
 - Minimum healthy percentage: **90%**
 - Instance warmup: **300 seconds**
 - Started the instance refresh.
- 9. **Verified Instance Refresh**
 - AWS started replacing the existing ASG instances with instances using the updated configuration.
 - Current screenshot shows:

- Refresh status: **In progress**
- Progress: **50%**
- **1 instance left to update**

10. Final Test — Still To Do

After the instance refresh reaches **100%**, generate CPU load on the ASG instances and monitor:

CloudWatch → CPUUtilization → Auto Scaling Group

When average CPU utilization rises sufficiently relative to the **70% target**, verify that Auto Scaling increases capacity, subject to the ASG maximum of **4 instances**.

Expected Result

Normal condition:

ALB → 2 EC2 instances

High CPU demand:

CPU increases → CloudWatch metric → Target Tracking Scaling Policy → ASG increases desired capacity → New EC2 instance(s) launched → Target Group health check → ALB distributes traffic

When CPU demand decreases, target tracking can subsequently reduce capacity while respecting the configured minimum capacity of **2 instances**.

Important Interview Explanation

You can explain it simply:

“I configured an Auto Scaling Group with minimum 2, desired 2 and maximum 4 instances. I created a target tracking scaling policy using average CPU utilization with a target value of 70%. When CPU demand increases, Auto Scaling can launch additional EC2 instances up to the maximum capacity. The new instances register with the target group, and the Application Load Balancer distributes traffic among healthy instances.”

aws [Search] [Alt+S] United States (N. Virginia) chandrashekar (986881030460) chandrashekar

EC2 > Instances

Instances (1/4) Info

Save filter sets Choose filter set Filter Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv
	i-05c11224e53674279	Running	t3.micro	3/3 checks passed	us-east-1b	-	-
ASG-Public-EC2	i-0261ae810cd2d35e5	Running	t3.micro	3/3 checks passed	us-east-1a	-	98.92.41.1
ASG-Private-EC2	i-0aa77243c76c0be42	Running	t3.micro	3/3 checks passed	us-east-1b	-	-
	i-09b82653e97b2d721	Running	t3.micro	3/3 checks passed	us-east-1b	-	-

i-09b82653e97b2d721

IP name: ip-10-0-4-216.ec2.internal

Answer private resource DNS name

Auto-assigned IP address

IAM role
EC2-CloudWatch-Monitoring-Role

IMDSv2
Required

Operator

ip-10-0-4-216.ec2.internal

Instance type
t3.micro

VPC ID
vpc-0978a6639ef05cc56 (ASG-VPC)

Subnet ID
subnet-00264d3e35fb7c9a2 (ASG-Private-Subnet-2)

Instance ARN
arn:aws:ec2:us-east-1:986881030460:instance/i-09b82653e97b2d721

Elastic IP addresses

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name
ASG-Web-ASG

Managed
false

aws [Search] [Alt+S] Ask Amazon Q United States (N. Virginia) chandrashekar (986881030460) chandrashekar

EC2 > Auto Scaling groups > ASG-Web-ASG

Auto Scaling group updated successfully

ASG-Web-ASG

ASG-Web-ASG Capacity overview

arn:aws:autoscaling:us-east-1:986881030460:autoScalingGroup:ca31981d-9657-4a19-805d-3991935110be:autoScalingGroupName/ASG-Web-ASG

Desired capacity: 2

Scaling limits: 2 - 4

Desired capacity type: Units (number of instances)

Status: At desired capacity

Date created: Wed Sep 23 2026 16:55:28 GMT+0530 (India Standard Time)

Details Integrations Automatic scaling Instance management Instance refresh Activity Monitoring Tags

Launch template

Launch template: lt-0506a0705168fce56 ASG-Web-LT

Version: 4

AMI ID: ami-0b6d9d3d33ba97d99

Security groups

Instance type: t3.micro

Security group IDs: sg-00d234feb6afb75a3

Owner: arn:aws:iam::986881030460:root

Create time: Wed Sep 23 2026 21:15:54 GMT+0530 (India Standard Time)

